Name: Ethan Hershey
Access ID: edh5292

Note on problem sizes: Tasks 2–3 use the full project problem (2 cars, 6 levels, 7 slots, 42 binary variables). Tasks 4–7 use the smaller example from Sturm (2023) Section 2.2 (1 car, 4 levels, 4 slots, 8 binary variables), as confirmed with the TA. Statevector simulation of 42 qubits requires $\approx 2^{42}$ complex amplitudes ($\sim$67 PB), which is infeasible on any available hardware.

Task 1 — Problem Understanding

Q1. The objective $f_1(\tilde{p}) = \tilde{p}^\top A\tilde{p}$ computes the squared ℓ_2-norm of the total load vector, where component t is the sum of all cars' charging levels at time slot t. Minimizing this penalizes uneven power draws: for a fixed energy budget, a flat profile gives a smaller squared sum than a peaked one (e.g., $2^2+2^2+2^2 = 12 < 1^2+2^2+3^2 = 14$). This directly minimizes peak grid load.

Q2. Setting $\tilde{p} = 0$ trivially minimizes f_1 to zero, but violates the energy constraints $C\tilde{p} = \tilde{e}$ requiring car_green to receive 8 units and car_orange to receive 12 units. The constraints exist precisely to exclude this trivial infeasible solution.

Q3. The penalty ρ converts the hard equality constraints into a soft penalty term $\rho\|C\tilde{p} - \tilde{e}\|^2$ added to the objective. If ρ is large enough, violating a constraint inflates the cost far beyond any feasible solution. If ρ is too small (e.g., $\rho = 3.0$), the optimizer may return a cheaper but infeasible solution where cars are undercharged. We require $\rho > 5.0$; we use $\rho = 5.1$.

Task 2 — Classical Optimization

The problem was formulated as a QCIO with 14 integer variables (one per car per time slot, range $[0,5]$) and two linear equality constraints enforcing the energy requirements. The objective $\tilde{p}^\top A\tilde{p}$ minimizes squared total load. Since `CplexOptimizer` is unavailable and `NumPyMinimumEigensolver` cannot handle 2^{42} states, the problem was solved by exhaustive enumeration over feasible integer schedules (a few thousand feasible points).

Optimal objective value: $f_1(p^*) = 58.0$ (matches target benchmark).

Car	$t=0$	$t=1$	$t=2$	$t=3$	$t=4$	$t=5$	$t=6$
car_green	2	0	3	3	0	0	0
car_orange	0	3	0	0	3	3	3
Total load	2	3	3	3	3	3	3

Table 1: Optimal charging schedule. Total energy: car_green = 8, car_orange = 12.

The solution is nearly flat (load 2–3 per slot), which is why $f_1 = 2^2 + 6 \times 3^2 = 4 + 54 = 58$.

Task 3 — QUBO Conversion

The QCIO was converted to QUBO in two steps using Qiskit's converter pipeline:

1. `LinearEqualityToPenalty` ($\rho = 5.1$): absorbs equality constraints as $\rho\|C\tilde{p} - \tilde{e}\|^2$ into the objective.
2. `IntegerToBinary`: encodes each integer $p \in \{0, \ldots, 5\}$ using $\lceil \log_2 6 \rceil = 3$ binary bits.

Number of integer variables: 14. Number of binary variables: $14 \times 3 = \mathbf{42}$.

The binary encoding is necessary because QAOA operates on qubits (binary), so all integer variables must be expressed in $\{0, 1\}$ form before a circuit can be built.

Effect of $\rho = 3.0$*:* For any schedule violating both energy constraints (e.g., all zeros), the penalty is $\rho\|\tilde{e}\|^2 = \rho(8^2 + 12^2) = 208\rho$. With $\rho = 5.1$, this equals 1060.8, always exceeding any feasible $f_1 \geq 58$. With $\rho = 3.0$, the penalty is only 624, which may not dominate, allowing the optimizer to return a solution where cars are undercharged.

Task 4 — QAOA Unoptimized ($p = 1$)

Using the 8-qubit small problem (Sturm 2023, Sec. 2.2), the QUBO was converted to an Ising Hamiltonian (`to_ising()`), giving offset = 34.4. A $p = 1$ QAOA circuit was built with `QAOAAnsatz` and evaluated on `AerSimulator` (statevector, 4000 shots) with fixed $\beta = \gamma = 1.0$.

Parameter	Value
p	1
β, γ	1.0, 1.0 (fixed)
Shots	4000
Expected QUBO cost	35.91
Classical optimum	6.0

Table 2: Unoptimized QAOA result.

The result is far from optimal because un-tuned gate angles assign nearly equal probability to all $2^8 = 256$ bitstrings. Most bitstrings correspond to infeasible, high-penalty solutions, so the expectation is dominated by these high-cost outcomes. Without optimized β and γ, the circuit cannot selectively amplify the probability of optimal bitstrings.

Task 5 — QAOA Optimization ($p = 1$)

QAOA parameters were optimized using COBYLA (max 200 iterations) with three random initializations: $\beta \sim \text{Uniform}[0, \pi]$, $\gamma \sim \text{Uniform}[0, 2\pi]$.

Seed	Init β	Init γ	Final β	Final γ	Final f
42	2.431	2.758	5.276	3.415	29.064
123	2.144	0.338	2.122	0.326	25.423
999	2.447	1.082	2.948	2.112	27.984

Table 3: COBYLA optimization results for $p = 1$.

Best result: $f = 25.42$ (seed 123). Classical optimum: 6.0.

All three seeds converge to values in the range 25–29, well above the optimum. COBYLA finds local minima quickly (26–33 iterations) because the landscape is highly non-convex: the many infeasible bitstrings dominate the expectation, creating a flat, noisy landscape with many local minima. The gap reflects both the limited expressivity of $p = 1$ and the stochastic noise in shot-based estimation.

Task 6 — Effect of Circuit Depth ($p = 2$)

Task 5 was repeated with $p = 2$ (4 parameters: $\beta_0, \beta_1, \gamma_0, \gamma_1$).

Seed	Final f	Iterations
42	29.864	42
123	34.136	37
999	23.968	39

Table 4: COBYLA results for $p = 2$.

Metric	$p = 1$	$p = 2$
Parameters	2	4
Best result	25.42 (seed 123)	23.97 (seed 999)
Worst result	29.06 (seed 42)	34.14 (seed 123)
Average	27.49	29.32

Table 5: $p = 1$ vs. $p = 2$ comparison.

Increasing p does not always improve results in practice. While the best result improved slightly (25.42 $\rightarrow$ 23.97), the average across seeds was worse for $p = 2$ (29.32 vs. 27.49), and seed 123 was significantly worse. The higher-dimensional parameter space is harder for COBYLA to navigate—a symptom of barren plateaus, where gradients vanish in large regions. In theory QAOA converges to the optimum as $p \rightarrow \infty$, but in practice the optimization difficulty grows faster than the benefit for moderate p with limited iterations.

Task 7 — Noisy Simulation

The best $p = 2$ circuit (seed 999, $\beta_0 = 2.44, \beta_1 = 0.54, \gamma_0 = 5.46, \gamma_1 = 4.71$) was run on two backends without re-optimizing parameters. The noisy backend was `FakeKolkataV2` (27 qubits), which models realistic IBM gate and readout errors. The circuit was transpiled to basis gates $\{$CX, ID, RZ, SX, X$\}$ with the backend's coupling map at optimization level 1.

Backend	Expected QUBO cost	Gap from optimum
Noiseless (`AerSimulator`)	24.37	18.37
Noisy (`FakeKolkataV2`)	30.92	24.92
Degradation	+6.55 (+27%)	—

Table 6: Noiseless vs. noisy simulation. Classical optimum = 6.0.

Noise degrades performance through three mechanisms: (1) SWAP overhead—limited qubit connectivity forces SWAP insertions, each decomposing into 3 CNOT gates and tripling two-qubit error accumulation; (2) depolarizing gate errors—each CNOT carries $\sim$0.1–1% error probability, compounding across the circuit; (3) readout errors—bit-flip noise in measurement distorts the estimated expectation. The combined effect pushes the output distribution toward uniform, washing out the amplitude amplification QAOA achieved during optimization. On real hardware this is even worse: SWAP routing adds more depth than in simulation, and gate errors are not just modeled but real.

Task 8 — Reflection

Q1. QAOA is a hybrid quantum-classical algorithm. The quantum processor evaluates $\langle\psi(\beta,\gamma)|H_P|\psi(\beta,\gamma)\rangle$ by preparing the parameterized state and sampling from it. The classical optimizer (COBYLA) receives this expectation value and proposes updated β and γ to reduce it, iterating until convergence. Neither component works alone: the quantum device provides the cost landscape evaluation, and the classical optimizer navigates it.

Q2. I would choose $p = 1$. Although $p = 2$ achieved a marginally better best-case result in noiseless simulation (23.97 vs. 25.42), the deeper circuit accumulates significantly more gate error in practice. Task 7 showed a 27% cost increase under realistic noise even at $p = 2$. On real hardware, the additional CNOT and SWAP gates from the extra layer would destroy any theoretical advantage. The shallower $p = 1$ circuit gives more reliable and interpretable results on current NISQ devices.

Q3. The number of binary variables (qubits) scales as $K \times T \times \lceil \log_2 L \rceil$, where K is cars, T is time slots, and L is charging levels. The full project problem (2 cars, 7 slots, 6 levels) already requires 42 qubits, which is too large for exact simulation and exceeds most available fake backends. On real hardware, more qubits means more SWAP routing and error accumulation. The circuit depth also grows quadratically with the number of Ising ZZ terms ($O(n^2)$), worsening the noise problem. The classical optimization landscape becomes exponentially harder with more parameters ($2p$ per repetition), further compounding the barren plateau issue. This is the core scaling challenge of NISQ-era QAOA.